Topology-Sensitive Failure Regimes in Agentic AI: A Cross-Benchmark Trace Audit with MAS Analyzer

Anonymous Authors[1]

Abstract

Multi-agent topologies are often evaluated as if additional agents were a general-purpose capability amplifier. We test that assumption with a trace-centered audit of MAS Analyzer runs across five benchmarks and seven coordination topologies. The completed corpus contains 34 benchmark–topology summary cells, 3,060 evaluated runs, 990 task-level metric rows, and 2,425 failed runs with trajectory and evaluation artifacts. The results show that topology effects are strongly benchmark-dependent: group-chat debate substantially improves StableToolBench, simple voting is strongest on BrowseComp, PlanCraft, and WorkBench, and orchestration barely moves Finance Agent despite large token costs. We add three trace diagnostics: tool-call Markov transitions, vote entropy, and first-critical-fault localization. Trace-level labels and clinical case studies reveal why benchmark identity largely determines the dominant failure regime, while topology mostly changes cost, communication, and the probability of escaping that regime. These results turn failure labels into reproducible trace triggers and repair hypotheses rather than post-hoc explanations.

1. Introduction

Agentic systems are increasingly evaluated through benchmark success rates, but terminal scores hide the process by which a run fails. A final answer can be wrong because retrieval locked onto a weak hypothesis, because a planner declared impossibility after a local shortage, because an API returned sparse evidence, or because a workflow never grounded a required person or email address. These are not interchangeable errors: each implies a different repair.

This paper studies whether multi-agent topology changes those failures. The question is deliberately narrower than "do more agents help?" We ask when topology changes success, when it only changes cost, and when the same benchmark-specific failure regime persists across coordination structures. We analyze a MAS Analyzer experiment suite spanning browsecomp, plancraft, stabletoolbench, workbench, and finance_agent. The evaluated systems include a single-agent baseline, flat voting, fully linked debate, group-chat debate, and three orchestrator variants.

Our central finding is that topology is not a universal intervention. The strongest topology differs by benchmark, and the largest success gain appears only on the tool-heavy StableToolBench. On several other benchmarks, richer coordination adds substantial token and communication overhead while leaving the dominant failure mode largely intact. We therefore frame failure modes as topology-sensitive but benchmark-anchored: topology can change the chance of escaping a failure regime, but the regime itself is usually induced by the benchmark's evidence and action structure.

The paper makes four contributions. First, it reports a cross-benchmark topology matrix over 34 completed benchmark–topology cells. Second, it codes 2,425 failed runs into operational failure regimes using evaluation JSON, trajectory markdown, and representative trace inspection. Third, it introduces a coordination-tax view that relates success lift to token and communication overhead, making expensive multi-agent gains easier to interpret. Fourth, it adds state-transition diagnostics: tool-call Markov transitions for tool-heavy tasks, vote entropy for consensus health, and first-critical-fault localization for tracing terminal failures back to their earliest visible bottleneck. We treat these diagnostics as falsifiable repair hypotheses; intervention verification is left to future work.

[1]Anonymous Institution, Anonymous City, Anonymous Region, Anonymous Country.AUTHORERR: Missing \icmlcorrespondingauthor.

Preliminary work. Under review by the International Conference on Machine Learning (ICML). Do not distribute.

2. Related Work

Our study builds on agent architectures that interleave reasoning, action, tool use, and feedback. ReAct showed how reasoning traces can be coupled with environment interaction (Yao et al., 2023); Toolformer studied learned tool invocation (Schick et al., 2023); Reflexion emphasized verbal feedback after failed attempts (Shinn et al., 2023); and AutoGen made multi-agent conversation a practical systems pattern (Wu et al., 2023). These works motivate the topologies we study, but they do not determine when coordination repairs failures rather than amplifying them.

Benchmark work has likewise exposed the brittleness of long-horizon execution. AgentBench evaluates LLMs as agents across interactive settings (Liu et al., 2023), and WebArena shows that realistic web tasks create failures not captured by static question answering (Zhou et al., 2023). Our contribution is complementary: rather than introduce a new benchmark, we use a common artifact format to compare how benchmark families interact with topology.

The distinction matters because many benchmark reports emphasize issue-level trends in final correctness. Our traces show that long-horizon drift often begins earlier: an evidence state becomes thin, a tool response becomes a bottleneck, or a grounded entity remains unresolved. Reflection and debate are useful only if they detect that boundary. When every agent inherits the same weak context, multi-agent discussion can become redundant confirmation rather than adversarial verification.

3. Corpus and Method

3.1. Experiment Scope

The suite contains five benchmarks and seven topology labels: single-agent, voting, fully linked debate, group chat debate, orchestrator without discussion, tree orchestrator, and discussion orchestrator. One aggregate summary cell is absent from the completed export, leaving 34 benchmark–topology cells. Each completed cell has 30 tasks and three repeated runs, yielding 3,060 evaluated runs. The task-level metric export contains 990 task rows over 33 cells and is used for token, latency, communication, handoff, and tool-call analysis.

3.2. Metrics

We report task success, stability across repeated runs, average token use, tool calls, communication count, and handoff count. For topology comparisons, we compute success lift relative to the single-agent baseline within the same benchmark and token multiplier relative to the same baseline. This gives a direct estimate of coordination tax: the extra inference budget spent for each topology-specific success gain.

3.3. Trace Artifacts

The audit is possible because the runtime standardizes intermediate agent outputs. Each answer-producing stage emits an answer artifact, summary, critique, revision request, confidence, unresolved issues, and evidence summary. The trajectory files also record task prompts, tool definitions, role assignments, visible peer packets, final reasons, and vote tallies when applicable. This structure is important methodologically: it lets us separate an unsupported final answer from an explicit blocked answer, and it lets us ask whether a topology introduced new evidence or merely recirculated the same artifact.

We treat the benchmark evaluator output as the outcome source of truth. The trace is then used to explain the failure path, not to override the evaluator. This distinction prevents fluent but wrong answers from being credited simply because they look coherent, and prevents honest blocked answers from being collapsed into generic model error.

3.4. Failure Coding

We code every failed run in the 34 completed cells, for 2,425 failures in total. Coding begins from the benchmark evaluator output, then uses the prediction string, final reason, trajectory markdown, vote tally, visible packets, tool outputs, and representative JSONL traces to assign one primary label. The labels are operational rather than universal. For example, PlanCraft failures are coded as premature impossibility when the run emits impossible after a local shortage; WorkBench failures are coded as entity grounding failure when a missing person, sender, email, or assignee blocks execution.

When multiple mechanisms appear in the same run, we use an earliest-bottleneck rule: the primary label is the first trace-visible condition that makes the benchmark objective unattainable under the observed run. If a sparse API response later leads to an unsupported answer, the run is coded as tool/interface fragility when the tool layer is the binding constraint, and as unsupported synthesis when the tools return usable evidence but the final synthesis overstates it. If a workflow later stalls because an email address was never grounded, the run is coded as entity grounding rather than generic workflow blocking.

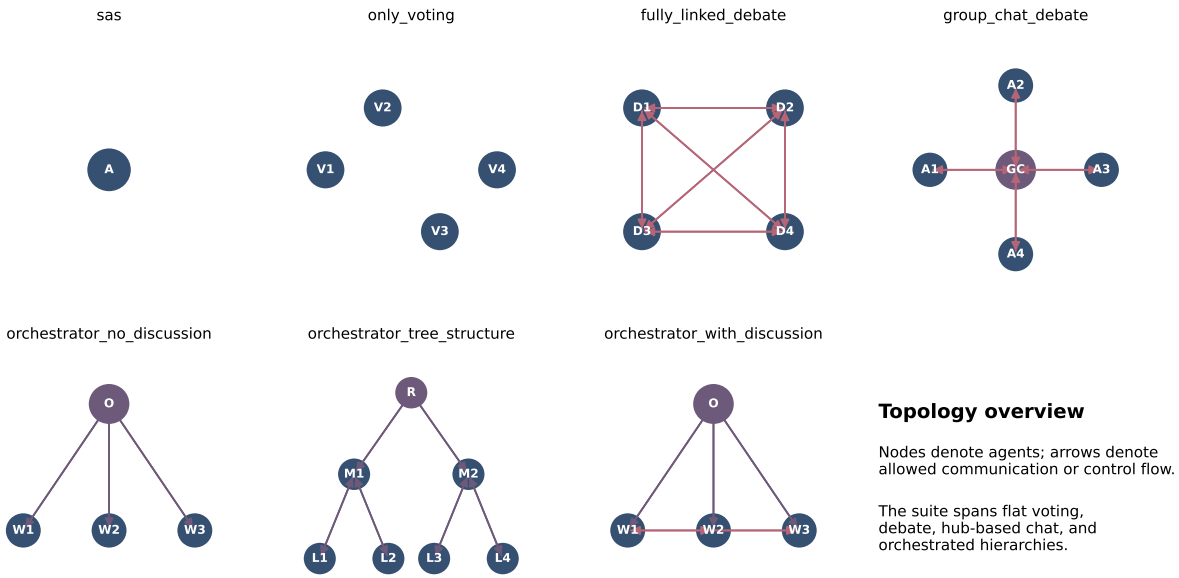

Figure 1. The evaluated topologies span single-agent execution, independent voting, connected debate, group chat, and orchestrated control.

This is a lightweight trace audit, not a multi-rater annotation study. Its purpose is diagnostic: to identify whether failures across topologies share the same trace-visible mechanism, and whether topology changes the distribution of those mechanisms. We therefore prefer labels with clear repair implications over labels that merely restate the benchmark outcome.

3.5. State-Transition Diagnostics

We add two programmatic diagnostics on top of the primary labels. For StableToolBench, we parse all non-communication tool-result events from run trace files and collapse each result into one of three states: OK, sparse, or error. We then estimate row-normalized Markov transitions separately for successful and failed runs. Table 1 gives the reproducible state rules. Observed status strings include completed, ok, and error; status values error, failed, or timeout are always treated as error.

For voting topologies, we compute Shannon entropy over the recorded vote tally. Let p_i be the share of recorded votes assigned to option i; then $H = -\sum_i p_i \log_2 p_i$, normalized by the maximum possible entropy for the number of observed options. This is a diagnostic of recorded consensus concentration, not a measure of correctness.

For the full failed-run corpus, we also compute a first-critical-fault label. This is not a claim of formal causal identification. It is a rule-based diagnostic attribution that maps each failed run to the earliest visible bottleneck class implied by evaluator output and trace artifacts: retrieval evidence gap, unsupported hypothesis, premature impossibility, tool/interface fault, entity grounding, workflow precondition, data-interface brittleness, or financial support gap. The first-critical-fault label differs from the primary failure label: the primary label explains the failed outcome, while first-critical-fault localizes the earliest trace-visible state that contaminates the rest of the run. We filter this diagnostic to the 34 completed cells in the experiment summary CSV. As a sanity check, we re-audited a deterministic stratified sample of 30 failed runs, six per

State	Rule	Examples
OK	Non-empty structured response, no error cue, and preview length at least 20 characters.	Valid JSON fields; populated records.
sparse	Empty or degenerate preview without an explicit error cue.	Empty list, empty object, null, no data found.
error	Status or preview contains interface-failure cues.	Cache miss, invalid parameters, exception, timeout, rate limit.

Table 1. Tool-result state rules used for StableToolBench Markov transitions. If multiple cues appear, error takes precedence over sparse.

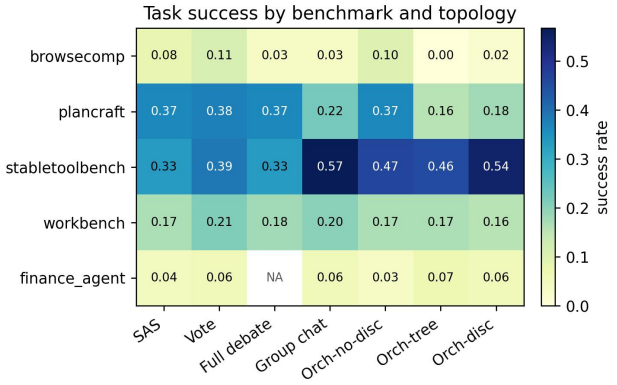

Figure 2. Task success by benchmark and topology. No topology dominates across benchmarks; topology effects are largest on StableToolBench and weakest on Finance Agent.

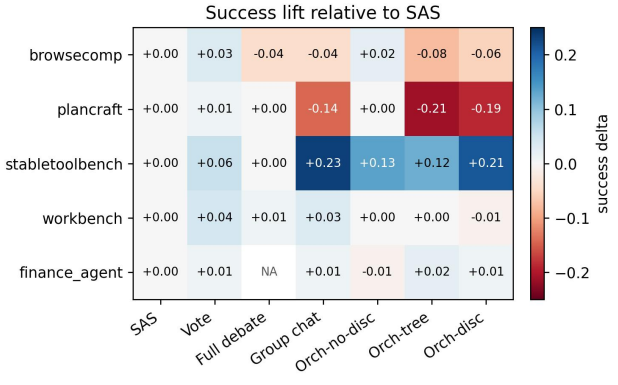

Figure 3. Success lift relative to the single-agent baseline. Positive effects concentrate in StableToolBench; several debate and orchestrator variants underperform the baseline on BrowseComp and PlanCraft.

Benchmark	Best topology	Best	SAS	Lift	Token mult.
browsecomp	only_voting	.111	.078	+.033	4.3×
plancraft	only_voting	.378	.367	+.011	3.9×
stabletoolbench	group_chat_debate	.567	.333	+.233	9.7×
workbench	only_voting	.211	.167	+.044	3.5×
finance_agent	orchestrator_tree_structure	.067	.044	+.022	8.4×

Table 2. Best topology by benchmark. The largest gain appears on StableToolBench, but it also requires the largest token multiplier among the winners.

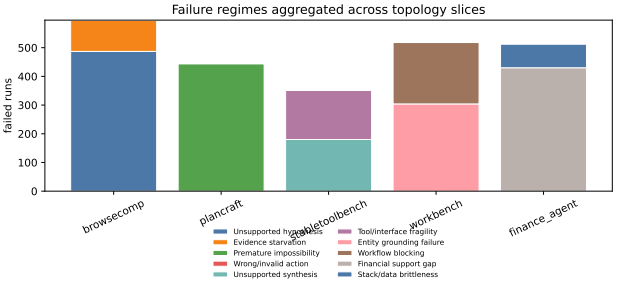

Figure 4. Failure labels aggregated across completed topology slices. The dominant failure regime is largely determined by benchmark family.

benchmark, and found 30/30 agreement with the rule output. This is a one-author reliability check rather than an independent multi-rater study.

4. Topology Does Not Have a Universal Winner

Figure 2 shows the central result. Voting is strongest on BrowseComp, PlanCraft, and WorkBench; group-chat debate is strongest on StableToolBench; and the tree orchestrator is strongest on Finance Agent. The best topology is therefore benchmark-dependent, not a general property of agent count.

Table 2 makes the trade-off explicit. StableToolBench gains 23.3 points over the single-agent baseline, while PlanCraft gains only 1.1 points despite using nearly four times as many tokens. Finance Agent is even more sobering: the best topology uses 8.4× the baseline tokens for a 2.2-point success lift. These results already suggest that coordination is useful only when it matches the benchmark's bottleneck.

5. Failure Regimes Are Benchmark-Anchored

Figure 4 aggregates all failed runs in completed cells. BrowseComp is dominated by unsupported hypotheses: 487 of 596 failed runs return an answer whose support chain is too weak, while 109 explicitly report evidence starvation. PlanCraft is the cleanest regime: 444 of 447 failures are premature impossibility. StableToolBench splits between unsupported synthesis (180) and tool/interface fragility (172), showing that tool-heavy tasks fail both at the interface and at the synthesis layer. WorkBench failures are mostly grounding related: 304 entity grounding failures and 214 broader workflow blocks. Finance Agent is dominated by financial support gaps (430 of 512), with 82 stack or data brittleness cases.

The important point is not that these labels are perfect. The important point is that topology rarely erases the benchmark's dominant regime. In BrowseComp, several coordination-heavy systems spend many more tokens but still fail by building around weak evidence. In PlanCraft, debate and orchestration do not prevent the local-to-global impossibility jump. In WorkBench, additional agents cannot execute a workflow if none of them resolves the missing entity. StableToolBench is the main exception: because its bottleneck often involves assembling partial tool evidence, group discus-

sion can actually change the run's chance of success.

Table 3 is the bridge from taxonomy to repair: each label is paired with a trace-visible state variable, a topology implication, and a diagnostic that can be converted into an assertion. The topology-conditioned heatmaps in Figure 5 support the same claim without aggregating away topology. Within most benchmarks, the darkest column remains stable across rows, while success rates and token costs vary. This is the empirical basis for saying that topology changes the probability of escaping a failure regime more often than it changes the regime itself.

6. Topology Changes Cost Before It Changes Failure

Figure 6 plots the economic side of the same story. Across non-baseline topologies, voting has the highest mean success lift (+3.1 points) with a lower mean token multiplier than discussion-heavy systems. Group-chat debate has high variance: it is excellent on StableToolBench but weak on BrowseComp and PlanCraft. The discussion orchestrator is especially costly, with negative mean lift across the task-level metric export.

This suggests a practical diagnostic: evaluate whether extra coordination changes the evidence state. If additional agents introduce genuinely new tool evidence or independent checks, the tax may be justified. If they merely paraphrase the same weak intermediate claim, the system pays for redundancy. The failure traces repeatedly show the latter pattern in BrowseComp and PlanCraft: discussion increases commitment or deliberation length without changing the decisive state variable.

The system-level averages sharpen this point. Among non-baseline systems in the task-level metric export, voting has the highest mean success lift (+3.1 points) while sending no inter-agent messages. By contrast, the discussion orchestrator averages roughly 27.8 communication events and 34.5 handoffs per task but has a negative mean lift. This does not imply that discussion is intrinsically bad. It implies that discussion must be evaluated by whether it changes the state variables that matter for the benchmark.

StableToolBench illustrates the positive case. It is the only benchmark where the best topology is explicitly discussion-heavy and the lift is large (+23.3 points). The benchmark's API workflows create a natural division of labor: one agent may map parameters, another may interpret a response, and a third may notice that a different endpoint is needed. BrowseComp and PlanCraft lack that same benefit in many traces. If all agents inherit the same weak snippet set or the same local shortage, discussion increases linguistic surface area without increasing search coverage or planning completeness.

7. State-Transition Diagnostics

7.1. Tool-Call Markov Transitions

The StableToolBench traces let us quantify the interface-fragility story directly. Figure 7 compares row-normalized transitions between coarse tool-result states for successful and failed runs. The largest separation is not the probability of remaining in an error state, which is high for both outcomes. Instead, it is the probability that an apparently usable tool state deteriorates on the next tool result. In successful runs, OK→sparse/error occurs with probability 0.184. In failed runs, the same deterioration occurs with probability 0.298, a 1.62× increase.

This matters because it reframes tool failure as a transition property rather than a terminal label. A single sparse response is not necessarily fatal; many successful runs also encounter imperfect tool outputs. The dangerous pattern is repeated deterioration without a strategy shift. Failed StableToolBench runs have a higher mean maximum adverse streak than successful runs (2.76 vs. 2.17), while using nearly the same mean number of tool results. The bottleneck is therefore not simply tool volume. It is whether the run can recover after the tool state becomes low-information.

7.2. Vote Entropy and Consensus

Figure 8 asks whether voting actually exposes disagreement. Across 1,236 runs with recorded vote tallies, consensus is often highly concentrated. In failed group-chat runs, the mean normalized vote entropy is only 0.084 and 91.6% of tallies contain a single surviving option. Fully linked debate is also concentrated: failed runs have mean normalized entropy 0.279 and a 69.7% single-option rate. Voting is less concentrated, but still far from adversarial: failed runs have mean normalized entropy 0.390 and a 58.6% single-option rate.

The key result is not that low entropy uniquely predicts failure. Successful runs also often have low entropy. Instead, vote entropy shows why consensus is an unsafe proxy for independent verification: agents can converge quickly around either a correct answer or a contaminated evidence state. This is the quantitative counterpart to the clinical PlanCraft example, where a feasible action appears in the artifact set but the final aggregation still promotes a terminal impossibility state.

Benchmark	Dominant trace mechanism	Topology implication	Repair-oriented diagnostic
BrowseComp	Evidence state remains thin while candidate answers become increasingly salient.	Debate can amplify weak hypotheses unless agents seek disconfirming evidence.	Unique evidence growth and candidate-disconfirmation checks before final vote.
PlanCraft	Local resource shortage is promoted to global impossibility.	More agents help only if one preserves the unexplored frontier.	Count alternate route expansions before impossible; require frontier-exhaustion proof.
StableToolBench	Sparse or schema-limited API responses constrain downstream synthesis.	Discussion helps when agents can combine complementary tool outputs.	Low-information response streaks, API coverage gaps, and synthesis support density.
WorkBench	Workflow cannot bind a required person, email, sender, or assignee.	Coordination is orthogonal unless an agent resolves the binding.	Unresolved entity count before first irreversible workflow action.
Finance Agent	Financial claims exceed verified filings, definitions, or extracted numeric support.	Orchestration adds cost but often cannot repair missing evidence.	Source coverage, period alignment, formula completeness, and citation-to-claim density.

Table 3. Failure regimes as repair hypotheses. Each label is tied to a trace-visible state variable and a topology-specific implication rather than treated as a generic model error.

7.3. First Critical Fault Localization

Figure 9 applies the same state-space view across all failed runs. The distribution is sharply benchmark-specific. In WorkBench, 364 of 518 failures (70.3%) localize first to entity grounding, with the remainder mostly workflow preconditions. In PlanCraft, 445 of 447 failures (99.6%) localize to premature impossibility. In StableToolBench, 286 of 352 failures (81.3%) localize to tool/interface faults. Finance Agent splits between financial support gaps (353 of 512, 68.9%) and data-interface brittleness (159 of 512, 31.1%). BrowseComp is dominated by unsupported hypothesis promotion (441 of 596, 74.0%), with 155 failures (26.0%) showing explicit retrieval evidence gaps.

These results sharpen the earlier failure taxonomy. The dominant failure is often not the last action. For WorkBench, the terminal symptom may be a missing calendar event, but the first critical fault is often unresolved identity or an unmet workflow precondition. For StableToolBench, the terminal symptom may be an unsupported synthesis, but the earliest visible bottleneck is often the tool state becoming sparse or schema-limited. For Finance Agent, the final answer may be fluent, blocked, or numerically specific, but the first critical fault is usually missing period-aligned financial support. This localization is why we frame repair as state-transition control rather than only better final-answer prompting.

Figure 10 summarizes the same point as a propagation view. Retrieval gaps and unsupported hypotheses feed contaminated evidence states, premature impossibility feeds invalid terminal states, and grounding failures feed unresolved action bindings. This is the picture behind the paper's repair claim: useful interventions should target the earliest state transition that contaminates the rest of the run.

Trace trigger	Programmatic metric	Expected failure regime
Repeated search with overlapping snippets	Unique-document growth across retrieval turns	Evidence starvation or unsupported hypothesis
Early impossible after a local shortage	Alternate route expansions before termination	Premature impossibility
Repeated sparse tool outputs	Low-information response streak length	Tool/interface fragility
Missing person or email before first action	Unresolved binding count	Entity grounding failure
Specific numerical answer with thin support	Support density and source disagreement	Financial support gap

Table 4. Trace-local triggers that can be converted into programmatic assertions. These metrics turn qualitative failure labels into testable state-transition checks.

7.4. Reproducible Trace Triggers

Table 4 makes the diagnostic proposal operational. Each row is a trace-local condition that can be monitored before the terminal answer. These triggers are not verified fixes. They are repair-oriented hypotheses: if the trigger is causal, then an intervention that changes the corresponding state variable should change the downstream failure rate.

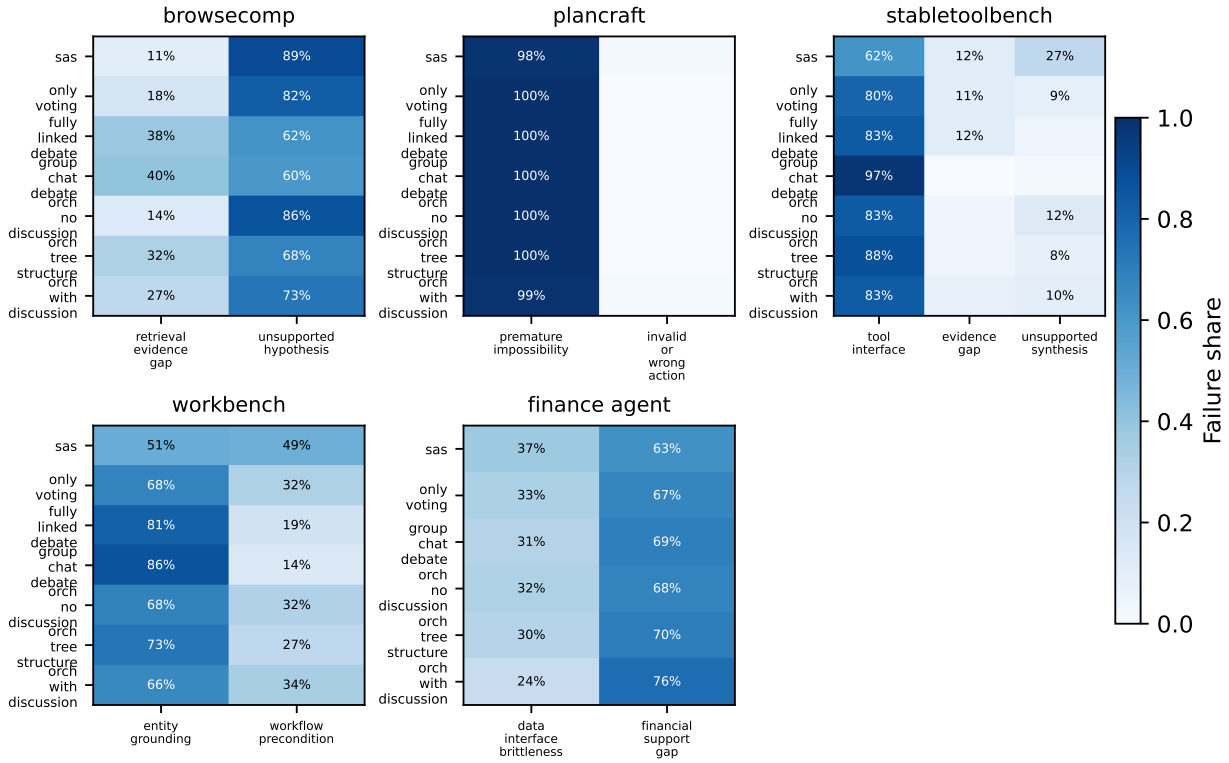

Figure 5. Topology-conditioned first-critical-fault shares. Rows are topologies and columns are first-fault classes within each benchmark. The dominant failure family is usually stable across topology even when success and cost change.

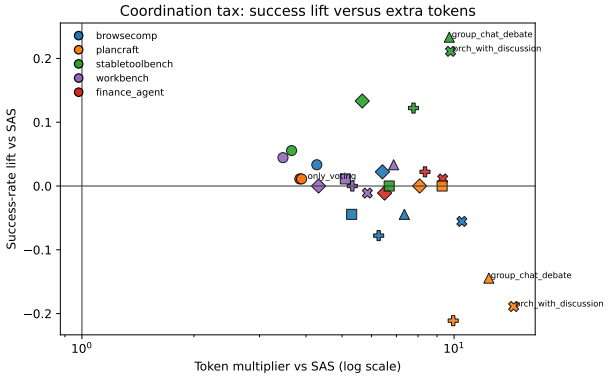

Figure 6. Coordination tax: success lift versus token multiplier relative to the single-agent baseline. StableToolBench is the only clear high-cost, high-gain case.

8. Clinical Trace Anatomy

8.1. BrowseComp: Hypothesis Lock-In

Task 769 asks for the learning institution satisfying five linked criteria, including a 2002 support event, a 2003 graduation ceremony, a 2022 plant-sampling article, a tribute event seven days later, and location in the country's capital. The reference answer is Queen Arwa University, but a fully linked debate run terminates with "Blocked: No direct evidence for the 2022 article (Criterion C) found yet." The final reason is no meaningful change, and the vote tally collapses to four variants of unknown.

The important detail is that the topology is structurally rich: it assigns a query decomposer, search strategist, document navigator, and answer synthesizer. Yet the trace shows the group converging on the same missing boundary rather than discovering the decisive document. The fatal pivot is not a hallucinated final answer; it is a stagnation state in which the system recognizes that Criterion C is unsupported but has no recovery mechanism beyond additional debate. Debate correctly avoids overclaiming, but it also fails to generate new evidence. This case therefore explains both sides of BrowseComp: unsupported answers are dangerous, but blocked answers can also reveal that coordination has not changed retrieval coverage.

8.2. PlanCraft: Local Shortage Becomes Global Impossibility

Task VAL0132 asks for the next action to craft pink dye. The inventory contains white dye and poppy. The benchmark guidance explicitly treats smelting as

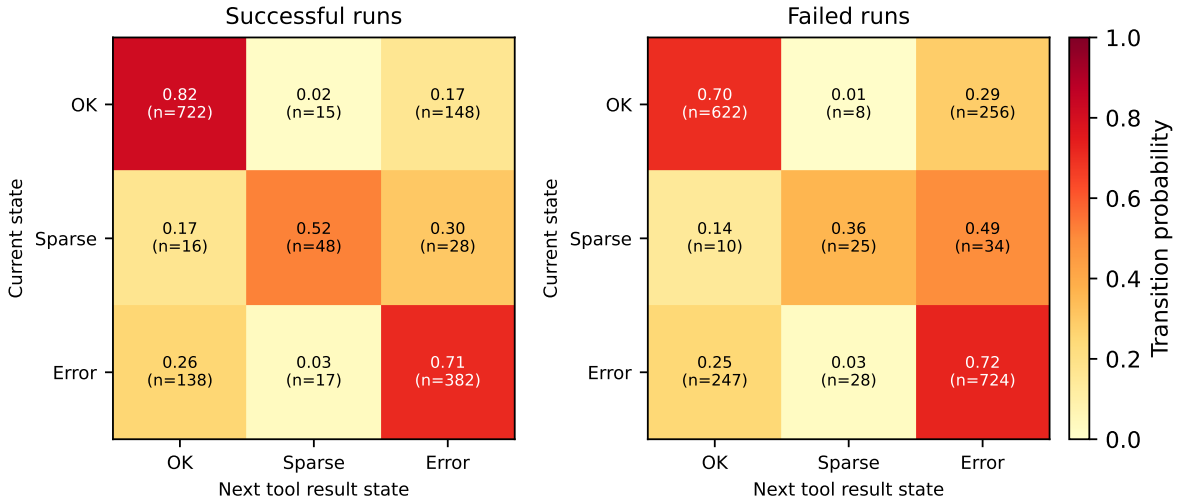

Figure 7. StableToolBench tool-result Markov transitions. Failed runs are more likely to transition from an OK tool result into an adverse sparse/error state on the next tool result.

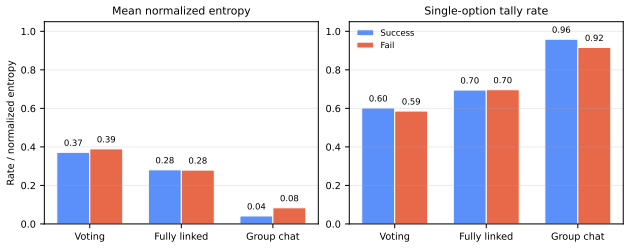

Figure 8. Recorded vote entropy by topology and outcome. Low entropy appears in both successes and failures, so consensus should not be treated as verification.

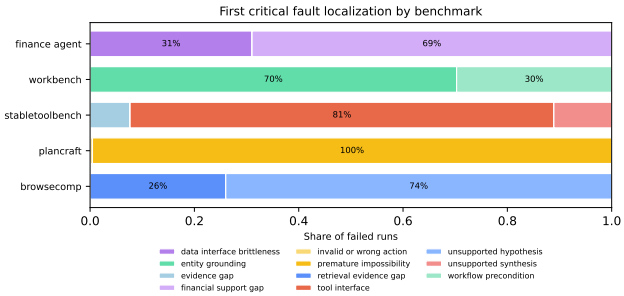

Figure 9. First critical fault localization over failed runs. The earliest visible bottleneck is benchmark-specific, supporting the claim that topology operates on top of benchmark-anchored failure regimes.

a valid action and says not to require furnace or fuel unless the prompt does. Several voter artifacts propose smelting the poppy into red dye, and one critique explicitly notes that marking the action impossible ignores the benchmark rule. Nevertheless, the final answer is impossible, with the reason that red dye is absent. The final reason is deterministic voting tiebreak.

This is a sharper failure than a generic planning miss. A valid route is visible in the artifact set, but the aggregation rule resolves a tie toward a terminal action. The fatal pivot is therefore a topology-mediated state transition: candidate actions include a feasible frontier-expanding step, yet finalization promotes the premature impossibility state. This case shows why voting is not automatically robust. It can reduce conversational amplification, but it still needs a domain-aware preference rule that ranks reversible frontier-expanding actions above irreversible impossible claims unless impossibility has been proven.

8.3. StableToolBench: API Schema Mismatch

Task 12204 requires airplane records ordered by name while also reporting passenger capacity and range. In a fully linked debate failure, the available ordered-airplanes endpoint sorts airplanes by name, while other endpoints expose different access patterns such as search by engine, brand, or single airplane id. The final vote favors a blocked answer: the tools cannot both order by name and include capacity/range in one request.

This trace is not merely a model reasoning failure. The tool surface creates a join problem: the desired answer requires fields and ordering guarantees that are not exposed by the same call. A stronger topology can help only if agents can decompose the operation into endpoint coverage, field recovery, and ordering validation. That is why StableToolBench is the benchmark where group chat plausibly pays for itself. The failure

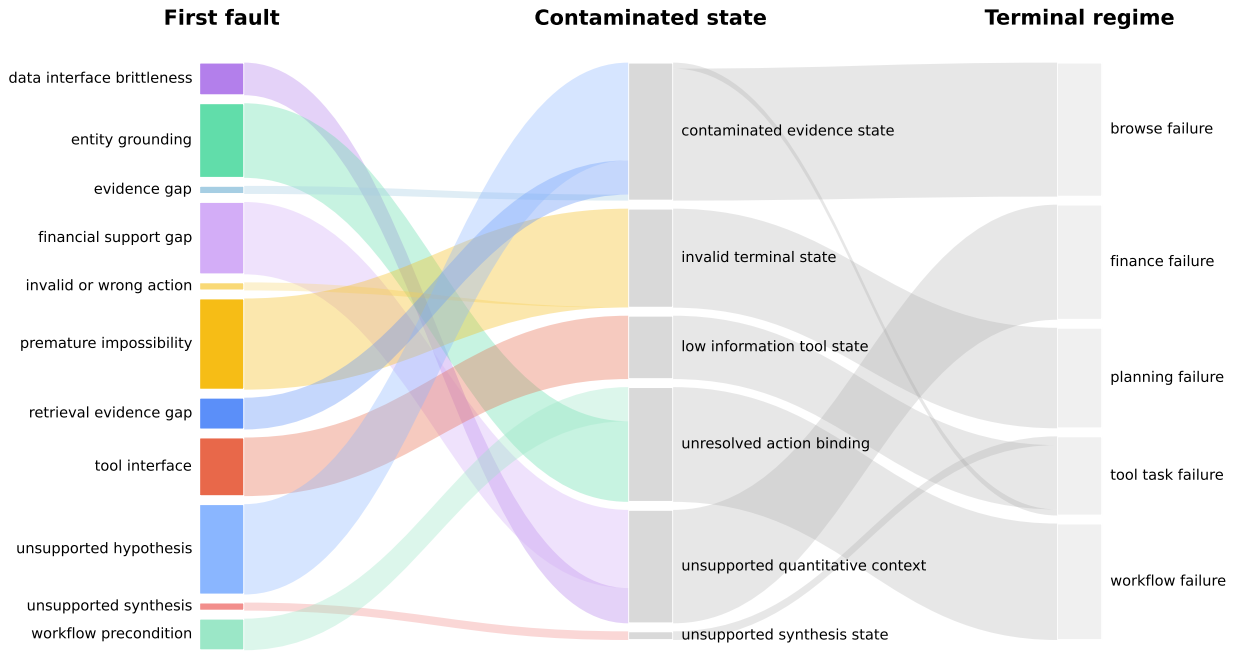

Figure 10. Aggregate diagnostic flow from first critical fault to contaminated state and terminal failure regime. The flow should be read as heuristic localization over traces, not as a formal causal graph.

mode is still interface fragility, but the repair is genuinely collaborative when different agents explore complementary API paths instead of repeating the same blocked endpoint.

8.4. WorkBench and Finance: Grounding Versus Support

Workbench task multi-domain 0 shows an operational grounding failure. The tool layer offers calendar operations and a company-directory lookup. The run repeatedly attempts to find Cameron Anderson's email address, then ends with "Meeting cannot be scheduled; no email found for Cameron Anderson or assignee." The final vote reaches consensus, but consensus is over an unresolved binding. No amount of scheduling logic can create the calendar event without the participant email.

Finance task 21 shows the parallel failure in evidential rather than operational form. The reference answer requires GBV per Nights and Experiences Booked for FY2022–FY2024. The trace includes EDGAR search, web search, parsing, and retrieval tools, but the final answer blocks because FY2023–FY2024 data remain unverified under rate limiting and HTML parsing constraints. Some intermediate artifacts contain partial calculations or placeholders, which is exactly the dangerous state for financial QA: the system can produce numeric-looking output before the evidence chain is complete. The fatal pivot to avoid is not only wrong arithmetic; it is allowing partial filings and period-misaligned evidence to support a precise final claim.

9. Toward Mechanistic Evaluation

The audit supports a state-space view of agentic failure. Each retrieval, tool call, or entity lookup updates the latent execution state. A later action may look reasonable while inheriting a corrupted state. BrowseComp failures often inherit a narrowed hypothesis space; PlanCraft failures inherit an invalid impossibility state; WorkBench failures inherit an unresolved entity binding; StableToolBench failures inherit a low-information tool state.

This framing turns failure labels into falsifiable mechanistic hypotheses. If evidence starvation is the driver, then forcing diverse retrieval or measuring unique evidence growth should change outcomes. If premature impossibility is the driver, then a formal frontier-exhaustion check should reduce false impossible actions. If tool fragility is the driver, causal interventions on tool responses should shift unsupported synthesis rates. The value of traces is that these hypotheses can be tested at the state-transition level, not only at the final-answer level.

The practical implication is that a stronger evaluation suite should log state changes, not only event counts. It is not enough to know that a tool was called; we need to know whether the call changed evidence coverage. It is not enough to know that several agents voted; we need to know whether the vote preferred reversible exploration over premature termination. And it is not enough to know that a financial answer contained citations; we need to know whether each numerical claim is aligned to the correct period, formula, and source.

10. Implications for System Design

The results argue against topology selection by leaderboard average alone. A useful multi-agent system should match coordination to failure regime. Tool-heavy tasks benefit from discussion when agents can combine partial external evidence. Search tasks need stagnation detection and adversarial evidence checks. Planning tasks need formal impossibility standards. Workflow tasks need early entity grounding before deeper orchestration. Finance tasks need support-sensitive answer policies that penalize fluent unsupported specificity. We present these as repair hypotheses supported by trace diagnostics, not as verified fixes.

The evaluation protocol should also report coordination tax. A topology that gains one or two points at 8–10× token cost may be valuable in a high-stakes setting, but it should not be reported as an unqualified improvement. Conversely, a cheap topology with modest gains may be a better default when the dominant failure mode is not repaired by discussion.

11. Threats to Validity

This study analyzes one experiment batch and one model family. The topology effects may change with stronger models, better tool interfaces, or different prompts. The failure labels are primary labels; many runs exhibit mixed mechanisms, especially tool fragility followed by unsupported synthesis. The coding procedure combines programmatic filtering with trace inspection rather than independent multi-rater annotation. The first-critical-fault re-audit is a one-author sanity check, not a full reliability study. Finally, one aggregate topology cell is absent from the completed summary export, so cost and task-level metric comparisons use the available completed cells.

12. Conclusion

Across five benchmarks and seven topology labels, multi-agent coordination is not a universal fix for agentic failure. StableToolBench benefits substantially from group-chat debate, but BrowseComp, PlanCraft, WorkBench, and Finance Agent show much smaller gains or expensive stagnation. The trace audit explains why: benchmark family largely determines the dominant failure regime, while topology mainly changes the cost and probability of escaping it. Future agent evaluation should therefore report topology-sensitive failure regimes, state-transition diagnostics, and coordination tax alongside terminal success.

Impact Statement

This paper studies failure in agentic AI systems with the goal of improving reliability and transparency. The positive impact is methodological: trace diagnostics can help developers separate unsupported answers from well-supported ones, distinguish tool failures from reasoning failures, and choose coordination structures with clearer evidence. The risk is that failure triggers could be used to construct adversarial tasks or to overfit systems to benchmark artifacts. In finance-style settings, fluent numerical answers with weak support chains are especially concerning because they can appear credible while failing a faithfulness test. We therefore recommend reporting failure taxonomies together with benchmark scope, uncertainty, and the limits of each diagnosis.

References

Liu, X. et al. Agentbench: Evaluating LLMs as agents. arXiv preprint arXiv:2308.03688, 2023. URL https://arxiv.org/abs/2308.03688.

Schick, T., Dwivedi-Yu, J., Dessì, R., Raileanu, R., Lomeli, M., Zettlemoyer, L., Cancedda, N., and Scialom, T. Toolformer: Language models can teach themselves to use tools. arXiv preprint arXiv:2302.04761, 2023. URL https://arxiv.org/abs/2302.04761.

Shinn, N., Cassano, F., Berman, A., and Gopinath, A. Reflexion: Language agents with verbal reinforcement learning. In Advances in Neural Information Processing Systems, 2023. URL https://arxiv.org/abs/2303.11366.

Wu, Q., Bansal, G., Zhang, J., Wu, Y., Li, B., Zhu, E., Jiang, L., Zhang, X., Wang, C., et al. Autogen: Enabling next-gen LLM applications via multi-agent conversation. arXiv preprint arXiv:2308.08155, 2023. URL https://arxiv.org/abs/2308.08155.

Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., and Cao, Y. React: Synergizing

reasoning and acting in language models. In International Conference on Learning Representations, 2023. URL https://arxiv.org/abs/2210.03629.

Zhou, S., Xu, F. F., Zhu, H., Zhou, X., Lo, R., Sridhar, A., Cheng, X., Bisk, Y., Fried, D., Alon, U., et al. Webarena: A realistic web environment for building autonomous agents. arXiv preprint arXiv:2307.13854, 2023. URL https://arxiv.org/abs/2307.13854.

A. Failure Coding Rubric

- Unsupported hypothesis: the run returns a candidate answer without sufficient supporting evidence.
- Evidence starvation: the run explicitly reports that necessary evidence was not recovered.
- Premature impossibility: the run emits impossible before the search frontier is exhausted.
- Tool/interface fragility: sparse, empty, inconsistent, or unavailable tool output is the primary bottleneck.
- Unsupported synthesis: tools return some evidence, but the final answer exceeds what the evidence supports.
- Entity grounding failure: a person, email, sender, recipient, user, or assignee cannot be resolved.
- Workflow blocking: execution stalls on an operational precondition not captured by entity grounding alone.
- Financial support gap: a financial answer is fluent or numerically specific but weakly supported.
- Stack/data brittleness: finance failures caused by interacting data, interface, and benchmark constraints.

B. First-Critical-Fault Rule Details

The first-critical-fault diagnostic is generated programmatically by the trace-state diagnostics script. The script first filters runs to benchmark–topology cells present in the experiment summary CSV; this excludes the incomplete aggregate cell from diagnostic counts. It then assigns the earliest bottleneck class using evaluator text plus selected trace checks:

- BrowseComp: explicit blocked, missing, unsupported, or no-evidence language maps to retrieval evidence gap; otherwise a concrete wrong answer maps to unsupported hypothesis.
- PlanCraft: answers containing impossible map to premature impossibility; other wrong actions map to invalid or wrong action.
- StableToolBench: tool error cues or adverse tool-result traces map to tool/interface fault; missing metadata or unresolved subqueries map to evidence gap; remaining failures map to unsupported synthesis.
- WorkBench: sparse company-directory email lookup results or explicit missing-email language map to entity grounding; remaining failures map to workflow precondition.
- Finance Agent: rate-limit, parsing, API, EDGAR, or other interface cues map to data-interface brittleness; remaining failures map to financial support gap.

We manually re-audited the deterministic stratified sample stored with the generated figure artifacts. The sample contains six failed runs per benchmark and matched the programmatic label in all 30 cases. This check verifies that the rules match visible trace evidence on the sample; it is not a substitute for independent multi-rater annotation.

C. Artifact Notes

The primary aggregate file is the experiment summary CSV. Task-level metrics are drawn from system-level and task-level metric CSV exports. Per-run diagnoses use evaluation JSON, trajectory markdown, and event-level trace JSONL files. The state-transition diagnostics are generated by the trace-state diagnostics script, which writes the Markov transition, vote-entropy, run-statistic, first-fault, topology-conditioned fault, re-audit sample, and propagation files under the figure artifact directory. The analysis excludes incomplete aggregate cells from topology-level summary claims.